

CELL 19 : STRICT KG-ONLY TOP-10 CHUNK RETRIEVER (CODE-WISE FLOW)

NO LLM · NO BM25 · NO embeddings · NO vector search
RANKING = seed_score x ex:strength x hop decay (100% graph-derived)

CODE (SECTION)

```
TOP_K = 10
CHUNK_SIZE = 900
MIN_CHUNK = 120
MAX_PER_DOC = 3
JACCARD = 0.25
SHINGLE_N = 5

_clean(t)
_shingles(t, n=SHINGLE_N)
_split_chunks(text, size=CHUNK_SIZE)
```

CODE (SECTION)

```
# ---- Chunk corpus, keyed by document URI ----
DOC_Q = PREFIXES + """
SELECT ?doc ?label ?ref ?content
WHERE {
  ?doc ex:documentContent ?content .
  OPTIONAL { ?doc rdfs:label ?label }
  OPTIONAL { ?doc ex:sourceReference ?ref }
}
"""

CHUNKS_BY_DOC = defaultdict(list)
for row in g.query(DOC_Q):
    uri = str(row.doc)
    title = str(row.label) if row.label
    else NODE_LABEL.get(uri, uri)
    ref = str(row.ref) if row.ref else uri
    for ch in _split_chunks(_clean(str(row.content))):
        CHUNKS_BY_DOC[uri].append({"content": ch,
                                     "title": title, "reference": ref, "doc_uri": uri})
```

CODE (SECTION)

```
# ---- Step 1: KG lexical node matching ----
seeds = lexical_fallback(QUERY_KG, limit=10)

# seeds -> list of tuples:
# (node_uri, label, seed_score, matched_tokens)
```

CODE (SECTION)

```
# ---- Step 2: SPARQL multi-hop traversal ----
edges, visited = traverse(seeds)

# edges -> list of dicts:
# {source, target, predicate, strength,
#  hop, seed_score}
# visited -> dict {node_uri: (min_hop, seed_score)}
```

CODE (SECTION)

```
# ---- Step 3: graph evidence per document ----
doc_evidence = defaultdict(float)
for e in edges:
    w = (e["strength"] / 10.0) * (HOP_DECAY ** (e["hop"] - 1)) * (e["seed_score"] / 100.0)
    doc_evidence[e["source"]] += w
    doc_evidence[e["target"]] += w

for uri, _lbl, _score, _p in seeds:
    doc_evidence[uri] = max(doc_evidence[uri], _score / 100.0)

scoped_docs = {d for d in doc_evidence
                 if d in CHUNKS_BY_DOC}
```

CODE (SECTION)

```
# ---- Step 4: rank chunks by doc evidence ----
scored = []
for uri in sorted(scoped_docs,
                  key=lambda d: doc_evidence[d],
                  reverse=True):
    ev = doc_evidence[uri]
    for c in CHUNKS_BY_DOC[uri]:
        scored.append({"c": c["content"],
                       "_score": round(ev, 4),
                       "_graph_evidence": round(ev, 4)})
scored.sort(key=lambda x: x["_score"], reverse=True)
```

CODE (SECTION)

```
# ---- Step 5: dedup + per-doc cap -> top K ----
chosen, seen_h, seen_sh, per_doc = [], [], [], defaultdict(int)
for c in scored:
    if len(chosen) >= TOP_K:
        break
    if per_doc[c["reference"]] >= MAX_PER_DOC:
        continue
    h = hashlib.md5(c["content"].encode()).hexdigest()
    if h in seen_h:
        continue
    sh = _shingles(c["content"])
    if sh and any([s in seen_sh for s in sh]):
        continue
    chosen.append(c); seen_h.add(h); seen_sh.append(sh)
    per_doc[c["reference"]] += 1
```

CODE (SECTION)

```
# ---- Response payload ----
response_kg = {
    "data": [
        {
            "content": c["content"],
            "title": c["title"],
            "reference": c["reference"]
        }
    ]
}
```

INPUT QUERY

QUERY_KG = ("What steps should be taken if an Engineering Supplier Integration Document (ESID) is not already available for a supplier?")

USES / DEPENDS ON

- PREFIXES, NODE_LABEL
- _tokens, HOP_DECAY, MAX_HOPS
- lexical_fallback(), traverse()
- from Cell 16

STEP 0: PARAMETERS & HELPER FUNCTIONS

(Text cleaning, shingle generation, sentence-boundary chunking)

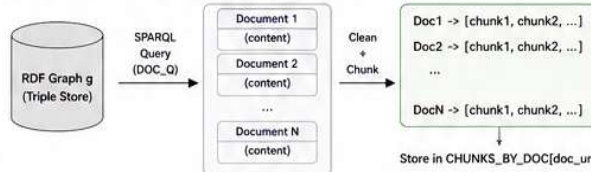
Helper	Purpose
_clean(t)	Remove unresolved links like [<doc:wiki:...>] and extra spaces
_shingles(t)	Create set of word shingles (n=SHINGLE_N) for dedup
_split_chunks(text)	Split cleaned text into sentence-boundary chunks (size=CHUNK_SIZE, no overlap, min length MIN_CHUNK)

Output

Constants
+
Helper Functions
ready

STEP 1: CORPUS BUILDING (From RDF Graph g)

Read document content from RDF graph 'g' and chunk it.

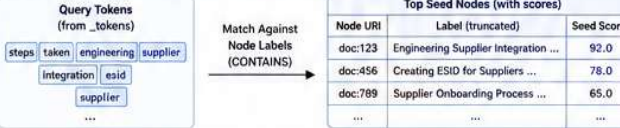


Output

- CHUNKS_BY_DOC (dict)
- { doc_uri: { (content, title, reference, doc_uri), ... } }
- N_CHUNKS (total chunks)
- Print corpus stats

STEP 2: KG LEXICAL NODE MATCHING (No LLM)

Tokenize query and match tokens against node labels (CONTAINS).

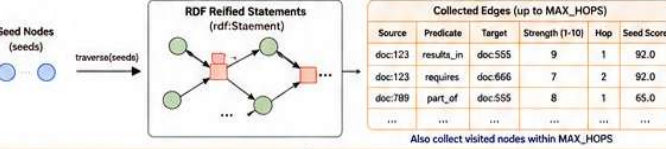


Output

- seeds = [(uri, label, score, tokens), ..., (top K)]
- If empty -> STOP

STEP 3: SPARQL MULTI-HOP TRAVERSAL OVER REIFIED STATEMENTS

Use SPARQL over rdf:Statement (iterative per hop up to MAX_HOPS).

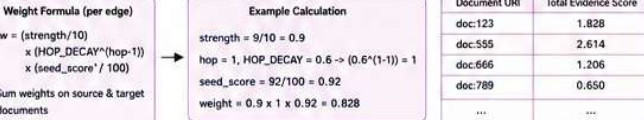


Output

- edges (list)
- visited (dict)
- Print counts

STEP 4: GRAPH EVIDENCE PER DOCUMENT

Aggregate evidence on BOTH endpoints of each edge.

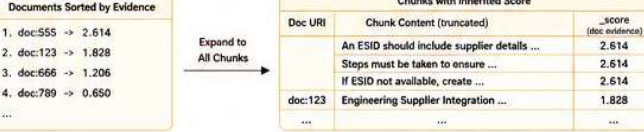


Output

- doc_evidence (dict{doc_uri:score})
- scoped_docs (set)
- Print count

STEP 5: RANK CHUNKS BY DOCUMENT'S GRAPH EVIDENCE

Every chunk inherits its document's evidence score.



Output

- scored (list of chunk dicts)
- Each dict has: content, title, reference, doc_uri, _score, _graph_evidence

STEP 6: DEDUPLICATION + PER-DOC CAP -> TOP 10

Remove duplicates and near-duplicates, cap per document, take Top 10.



Output

- chosen (list, len<=TOP_K)
- Top 10 chunk dicts after dedup & per-doc cap

STEP 7: RESPONSE PAYLOAD (No LLM)

Return only content, title, reference for top 10 chunks.



Output Variable

response_kg (dict)